

Parallel Quadratic Error Matrix Mesh Simplification

313551124 陳禹辰
NYCU
Hsinchu, Taiwan
yuchen.cs13@nycu.edu.tw

313551138 劉昶翰
NYCU
Hsinchu, Taiwan
hankliu755@gmail.com

313553003 張綺恩
NYCU
Hsinchu, Taiwan
kieann.cs13@nycu.edu.tw



Figure 1: Sample image of mesh simplification from [1]

1 INTRODUCTION

Mesh simplification is a crucial technique in computer graphics and 3D modeling that reduces the complexity of 3D meshes while preserving their overall shape and appearance. It involves decreasing the number of vertices, edges, and faces in a mesh, resulting in lighter models that are easier to process and render. This process is particularly important in applications such as video games, virtual reality, and simulations, where performance and real-time rendering are essential.

2 PROBLEM STATEMENT

However, as 3D models grow in size and complexity, traditional sequential algorithms for mesh simplification face significant limitations. These methods process mesh vertices and faces in a linear fashion, leading to inefficiencies resulting in long wait time in simplifying the mesh structure, especially with large-scale meshes with lots of vertices to compute.

Thus in this project, our method revolves around the QEM mesh simplification method [1]. This method includes several key steps including:

- (1) Computing the Q matrices for all initial vertices.
- (2) Selecting all valid vertex pairs.
- (3) Compute the optimal contraction target $\bar{v}$ for each valid pair (v_1, v_2) , where the contraction cost is given by the error $\bar{v}^T(Q_1 + Q_2)$.
- (4) Place all pairs in a heap keyed by cost, with the minimum cost pair at the top.
- (5) Iteratively remove the least cost pair (v_1, v_2) from the heap, contract the pair, and update the costs of all valid pairs involving v_1 .

Our hypothesis is that the main bottleneck stems from the time needed to compute the Q error for all the vertices. In addition to the iterative selection and collapse of edges.

3 APPROACHES

To address the above mentioned issues, we propose two parallelization methods:

3.1 Approach 1: Parallel QEM Error Computation and Edge Selection

- (1) **Parallel QEM Computation:** Distribute the computation of QEM errors across multiple threads or processors to calculate the Q matrices for all vertices concurrently.
- (2) **Parallel Sorting:** After all QEM errors are computed, the cost of the edge of valid pairs are placed in the heap. Thus the optimal solution is to parallelize the selection. An initial survey show the method of [2]. Where the heap is maintain by different threads.

3.2 Approach 2: Submesh-Based QEM Calculation

- (1) **Mesh Partitioning:** Divide the mesh into smaller sub-meshes. In our implementation we plan to first use pre-existing libraries such as, RXMesh [3] or ParMETIS [4]
- (2) **Independent QEM Computation:** Allow each submesh to independently compute its Q matrices and valid pairs in parallel, maximizing the use of computational resources.
- (3) **Sub-Mesh Simplification:** Perform local contraction operations within each sub-mesh based on the least cost pairs identified, taking advantage of parallel processing.
- (4) **Merge Mesh:** Reassemble the simplified mesh by merging the processed sub-meshes, ensuring that the topological integrity and quality of the mesh are maintained.
- (5) **Global Optimization:** After local contraction, the merged edges mesh would result in the i-regular shape in the mesh. Thus we would perform a final pass of QEM edge contraction to resolve the problem.

By implementing these parallelization methods, we can hope to significantly enhance the efficiency of the mesh simplification process while preserving the quality of the resulting mesh.

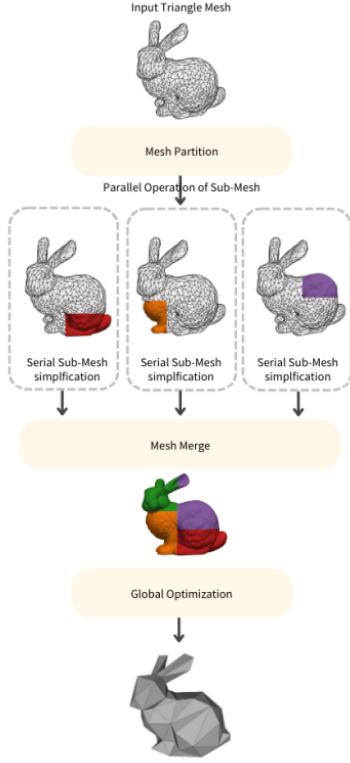


Figure 2: Diagram of Approach II

4 LANGUAGE SELECTION

In this project, we will utilize OpenMP, CUDA, and Pthreads as our primary parallel computing frameworks to efficiently implement the QEM mesh simplification methods.

4.0.1 OpenMP. OpenMP enhances thread management and workload distribution through straightforward directives, enabling easy implementation of parallelism in the code. This approach improves the parallel computation of QEM errors and vertex pair sorting on multi-core CPUs, leading to better resource utilization and faster computation times.

4.0.2 Pthread. Pthreads provides fine-grained control over threading and resource management, allowing for optimization of specific code segments. By combining Pthreads with OpenMP and CUDA, we achieve optimal utilization of both shared-memory and distributed-memory architectures, maximizing performance in the mesh simplification process.

4.0.3 CUDA. CUDA accelerates QEM calculations by leveraging GPU capabilities for parallel processing. This allows the handling of extensive data operations, such as data processing and matrix calculations, much more efficiently than traditional CPU execution, resulting in significant reductions in processing time.

5 RELATED WORK

The field of mesh simplification has advanced notably with Quadric Error Metrics (QEM). Garland and Heckbert’s pioneering work established foundational methods for efficient surface reduction while maintaining visual quality[1]. Building upon this, Hoppe introduced view-dependent refinement techniques that adapt mesh detail based on the viewer’s perspective, enhancing visual fidelity[5].

Adaptive simplification strategies have also emerged, such as the vertex clustering method by Pengdong et al., which preserves topology while improving computational efficiency[6]. Shaffer and Garland focused on adaptive simplification of massive meshes, emphasizing performance optimization[7]. Additionally, Franc and Skala examined how mesh simplification can consider model appearance, showcasing the importance of aesthetic factors in simplification[8].

In the realm of parallel computing, Low and Tan’s vertex-clustering method demonstrated effective model simplification[9]. Hoppe’s work on progressive meshes further illustrates techniques that balance complexity and performance[10]. MacNeice et al. introduced PARAMESH, a community toolkit for parallel adaptive mesh refinement, underscoring the relevance of parallel strategies[11]. Furthermore, DeCoro and Tatarchuk’s real-time GPU-based simplification methods reflect a growing trend toward utilizing parallel hardware for enhanced performance in interactive applications[12].

These studies collectively highlight the diverse strategies in mesh simplification, particularly those integrating adaptive methods and parallel processing to improve efficiency and quality.

6 EXPECTED RESULTS

We expect that the implementation of parallel QEM mesh simplification methods will yield a significant reduction in computational time while maintaining the geometric integrity of the simplified mesh. Additionally, we anticipate that the resulting meshes will exhibit high visual fidelity, with minimal loss in detail, ensuring that they remain suitable for applications in computer graphics and visualization.

7 TIMETABLE

Date	Task
Week 1 (10/20-11/02)	Literature review and initial setup
Week 2 (11/03-11/16)	Implement baseline QEM in C++
Week 3 (11/17-11/30)	Parallelize QEM using OpenMP
Week 4 (12/01-12/07)	Optimize and debug the parallel version
Week 5 (12/08-12/14)	Performance evaluation and analysis
Week 6 (12/15-12/22)	Prepare final report and presentation

Table 1: Project Timetable

REFERENCES

- [1] Michael Garland and Paul S. Heckbert. Surface simplification using quadric error metrics. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 209–216, New York, NY, USA, 1997. ACM.
- [2] Sushil Prasad and N. Deo. Parallel heap: improved and simplified. pages 448 – 451, 04 1992.

- [3] Ahmed H. Mahmoud, Serban D. Porumbescu, and John D. Owens. RXMesh: A GPU mesh data structure. *ACM Transactions on Graphics*, 40(4):104:1–104:16, August 2021.
- [4] George Karypis and Vipin Kumar. A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing*, 20(1):359–392, 1998.
- [5] Hugues Hoppe. View-dependent refinement of progressive meshes. In *Proceedings of the 24th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '97)*, pages 189–198. ACM Press/Addison-Wesley Publishing Co., 1997.
- [6] G. Pengdong, L. Ameng, L. Yongquan, W. Jintao, L. Nan, and Y. Wenhua. Adaptive mesh simplification using vertex clustering with topology preserving. In *2008 International Conference on Computer Science and Software Engineering*, pages 971–974, Wuhan, China, 2008. IEEE.
- [7] Eric Shaffer and Michael Garland. Efficient adaptive simplification of massive meshes. In *Proceedings of the Conference on Visualization '01 (VIS '01)*, pages 127–134. IEEE Computer Society, 2001.
- [8] Martin Franc and Václav Skala. Mesh simplification with respect to a model appearance. In *Proceedings of the 22nd Spring Conference on Computer Graphics (SCCG '06)*, pages 139–146, New York, NY, USA, 2006. Association for Computing Machinery.
- [9] Kok-Lim Low and Tiow-Seng Tan. Model simplification using vertex-clustering. In *Proceedings of the 1997 Symposium on Interactive 3D Graphics (I3D '97)*, pages 75–ff, New York, NY, USA, 1997. Association for Computing Machinery.
- [10] Hugues Hoppe. Progressive meshes. In *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH '96*, page 99–108, New York, NY, USA, 1996. Association for Computing Machinery.
- [11] Peter MacNeice, Kevin M. Olson, Clark Mobarry, Rosalinda de Fainchtein, and Charles Packer. Paramesh: A parallel adaptive mesh refinement community toolkit. *Computer Physics Communications*, 126(3):330–354, 2000.
- [12] Christopher DeCoro and Natalya Tatarchuk. Real-time mesh simplification using the gpu. In *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games (I3D '07)*, pages 161–166, New York, NY, USA, 2007. Association for Computing Machinery.

OTHER REFERENCES

- [1] <https://github.com/mgruppi/parallelmeshsimplification/tree/master>
- [2] <https://github.com/Zielon/ParallelQSLim>